

USE CASE 1 — LOAD AND CLEAN CHICAGO CRIME DATA

Name: Bhimireddy Bharath kumar Reddy

Email: bhimireddy.bharath99@gmail.com

Objective: Build a data ingestion pipeline to read, clean, process, and insert Chicago crime data into SQLite3.

Tasks

1. Load the Dataset

1.1 Inspect the First 10 Rows, Schema and Data Types

Description: Load the Chicago crime CSV dataset into a Pandas DataFrame and inspect the first 10 records along with the schema and data types.

Code:

```
def inspect_dataset(data):
    show_subsection("1.1 DISPLAY FIRST 10 ROWS AND DATA TYPES")
    print("\nFirst 10 records:")
    print(data.head(10))
    print("\nData types:")
    print(data.dtypes)
```

Output:

```
First 10 records:
   id case_number  ... longitude location
0  10000000    SS442871  ... -87.673423 (41.71608165, -87.67342345)
1  10000001    WW348136  ... -87.626794 (41.94851207, -87.62679436)
2  10000002    GG525832  ... -87.629846 (41.96394084, -87.62984598)
3  10000003    NC685131  ... -87.607711 (41.77004756, -87.60771146)
4  10000004    YT673638  ... -87.602075 (41.72772815, -87.60207475)
5  10000005    CR348066  ... -87.582994 (41.85177018, -87.58299423)
6  10000006    XE039726  ... -87.522090 (41.98069888, -87.52208991)
7  10000007    QK559272  ... -87.621489 (41.74574749, -87.62148947)
8  10000008    YD115267  ... -87.709245 (41.85296539, -87.70924486)
9  10000009    DI878822  ... -87.516485 (41.72356216, -87.51648472)
```

```
[10 rows x 22 columns]
```

```
Data types:
id                int64
case_number       str
date              str
block            str
iucr_code         int64
primary_type      str
description       str
location_desc     str
arrest           bool
domestic         bool
beat_num         int64
district_code     int64
ward_no          float64
community_code    float64
fbi_code         str
x_coordinate      float64
y_coordinate      float64
year             int64
date_of_update    str
latitude         float64
longitude         float64
location         str
dtype: object
```

```
Data Schema:
<class 'pandas.DataFrame'>
RangeIndex: 2000 entries, 0 to 1999
Data columns (total 22 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   id                    2000 non-null  int64
 1   case_number           2000 non-null  str
 2   date                  2000 non-null  str
 3   block                 2000 non-null  str
 4   iucr_code             2000 non-null  int64
 5   primary_type          2000 non-null  str
 6   description           2000 non-null  str
 7   location_desc         1906 non-null  str
 8   arrest                2000 non-null  bool
 9   domestic              2000 non-null  bool
10   beat_num              2000 non-null  int64
11   district_code         2000 non-null  int64
12   ward_no               1963 non-null  float64
13   community_code        1964 non-null  float64
14   fbi_code              2000 non-null  str
15   x_coordinate          1933 non-null  float64
16   y_coordinate          1936 non-null  float64
17   year                  2000 non-null  int64
18   date_of_update        2000 non-null  str
19   latitude              1927 non-null  float64
20   longitude              1936 non-null  float64
21   location              1926 non-null  str
dtypes: bool(2), float64(6), int64(5), str(9)
memory usage: 557.5 KB
None
```

Insights:

- The first 10 rows confirm that the Chicago crime dataset has been successfully loaded into a Pandas DataFrame.
- The displayed schema identifies the available columns and their corresponding data types.
- The dataset contains a combination of numerical, categorical, and date-related fields.

1.2 How Many Columns and Rows Are There?

Description: Determine the total number of rows and columns available in the dataset.

Code:

```
def dataset_size(data):  
    show_subsection("1.2 NUMBER OF ROWS AND COLUMNS")  
    row_count, column_count = data.shape  
    print("Number of rows      :", row_count)  
    print("Number of columns   :", column_count)
```

Output:

```
Number of rows      : 2000  
Number of columns   : 22
```

Insights:

- The DataFrame contains 2,000 rows and 22 columns at this stage of the dataset.
- The number of rows represents the available crime records.
- Knowing the dataset dimensions helps understand the size and scope of the data being processed.

2. Clean the Dataset

2.1 Convert Date Column into Datetime

Description: Convert the existing date column into Pandas datetime format so that it can be used for date-based analysis and feature generation.

Code:

```
show_subsection("2.1 CONVERT DATE COLUMN INTO DATETIME")  
date_column = None  
for col in data.columns:  
    if col.strip().lower() == "date":  
        date_column = col  
        break  
if date_column is not None:  
    data[date_column] = pd.to_datetime(  
        data[date_column],  
        errors="coerce"  
    )  
    print("Date column converted successfully.")  
    print("Date data type:", data[date_column].dtype)  
else:  
    print("Date column was not found.")  
"
```

Output:

```
Date column converted successfully.  
Date data type: datetime64[us]
```

Insights:

- The date column was successfully converted into datetime format.

- Converting the date allows year, month, and day-of-week information to be extracted.
- Invalid date values are handled using the appropriate conversion method.
- The converted date field can now be used reliably for further analysis.

2.2 Identify and Handle Missing Values

Description: Identify missing values in the dataset and handle them appropriately.

Code:

```
show_subsection("2.2 IDENTIFY AND HANDLE MISSING VALUES")
missing_before = data.isna().sum()
print("\nMissing values before cleaning:")
print(missing_before)
# Columns where text values are normally expected
text_columns = data.select_dtypes(
    include=["object", "string"]
).columns
for col in text_columns:
    data[col] = data[col].fillna("UNKNOWN")
# Numeric columns are kept numeric.
# Missing numeric values are replaced with 0.
numeric_columns = data.select_dtypes(
    include=[np.number]
).columns
for col in numeric_columns:
    data[col] = data[col].fillna(0)
print("\nMissing values after cleaning:")
print(data.isna().sum())
"
```

Output:

Missing values before cleaning:	Missing values after cleaning:
id	0
case_number	0
date	0
block	0
iucr_code	0
primary_type	0
description	0
location_desc	0
arrest	0
domestic	0
beat_num	0
district_code	0
ward_no	0
community_code	0
fbi_code	0
x_coordinate	0
y_coordinate	0
year	0
date_of_update	0
latitude	0
longitude	0
location	0
dtype: int64	dtype: int64

Insights:

- Missing values were identified across the dataset before cleaning.
- Missing text values were replaced with UNKNOWN.
- Missing numerical values were replaced with 0.
- The dataset was checked again after cleaning to verify the remaining missing values.

2.3 Standardize Categorical Data

Description: Standardize categorical fields by removing unnecessary space and converting text values to uppercase.

Code:

```
show_subsection("2.3 STANDARDIZE CATEGORICAL DATA")
categorical_columns = data.select_dtypes(
    include=["object", "string"]
).columns
for col in categorical_columns:
    data[col] = (
        data[col]
        .astype("string")
        .str.strip()
        .str.upper()
    )
print("Categorical fields standardized successfully.")
return data
```

Output:

```
Categorical fields standardized successfully.
```

Insights:

- Categorical values were standardized to provide a consistent format.
- Leading and trailing spaces were removed from text values.
- Text values were converted to uppercase.
- Standardization improves consistency during filtering, grouping, and analysis.

3. Generate New Features

3.1 Extract Year

Description: Extract the year from the converted date column and create a new Year feature.

Code:

```
show_subsection("3.1 EXTRACT YEAR")
data["Year"] = data[date_column].dt.year
print("Year feature created.")
print(data[["Year"]].head(10))
```

Output:

```
Year feature created.
   Year
0  2022
1  2015
2  2021
3  2019
4  2017
5  2017
6  2015
7  2018
8  2021
9  2018
```

Insights:

- A new Year feature was generated from the crime date.
- The year feature can be used to analyze crime trends over different years.

- This feature will also be useful for creating time-based visualizations.

3.2 Extract Month

Description: Extract the month from the date column and create a new Month feature.

Code:

```
show_subsection("3.2 EXTRACT MONTH")
data["Month"] = data[date_column].dt.month
print("Month feature created.")
print(data[["Month"]].head(10))
```

Output:

```
Month feature created.
   Month
0       3
1       6
2       3
3       4
4       9
5       7
6      11
7       7
8       1
9       9
```

Insights:

- A new Month feature was created from the date field.
- The month information can be used to identify seasonal or monthly crime patterns.
- It provides an additional time dimension for analysis.

3.3 Extract Day of Week

Description: Extract the day of the week from the crime date.

Code:

```
show_subsection("3.3 EXTRACT DAY OF WEEK")
data["DayOfWeek"] = data[date_column].dt.day_name()
print("DayOfWeek feature created.")
print(data[["DayOfWeek"]].head(10))
print("\nNew features generated successfully.")
return data
```

Output:

```
DayOfWeek feature created.
   DayOfWeek
0    Friday
1  Saturday
2 Wednesday
3    Friday
4  Thursday
5 Wednesday
6  Thursday
7    Sunday
8    Monday
9    Friday
```

Insights:

- A new DayOfWeek feature was generated.

- This allows crime records to be analyzed based on weekdays.
- The feature can help identify differences in crime occurrence across days of the week.

4. Use NumPy

4.1 Calculate Missing Value Percentage

Description: Calculate the percentage of missing values for each column using NumPy.

Code:

```
show_subsection("4.1 CALCULATE MISSING VALUE PERCENTAGE")
total_rows = len(data)
if total_rows > 0:
    missing_counts = data.isna().sum().to_numpy()
    missing_percent = (
        missing_counts / total_rows
    ) * 100
    missing_report = pd.Series(
        missing_percent,
        index=data.columns
    )
    print("Percentage of missing values per column:\n")
    print(missing_report)
else:
    print("Dataset contains no rows.")
    missing_report = pd.Series(dtype=float)
```

Output:

```
Percentage of missing values per column:
id                0.0
case_number       0.0
date              0.0
block             0.0
iucr_code         0.0
primary_type      0.0
description        0.0
location_desc     0.0
arrest            0.0
domestic          0.0
beat_num          0.0
district_code     0.0
ward_no           0.0
community_code    0.0
fbi_code          0.0
x_coordinate       0.0
y_coordinate       0.0
OriginalYear      0.0
date_of_update    0.0
latitude          0.0
longitude         0.0
location          0.0
Year              0.0
Month             0.0
DayOfWeek         0.0
dtype: float64
```

Insights:

- The missing-value percentage was calculated for every column.
- This helps identify fields that require additional cleaning or attention.
- NumPy was used to efficiently perform the percentage calculation.

4.2 Columns with More Than 50% Missing Values

Description: Identify columns where more than 50% of the values are missing.

Code:

```
show_subsection("4.2 COLUMNS WITH MORE THAN 50% MISSING VALUES")
high_missing = missing_report[
    missing_report > 50
]
if high_missing.empty:
    print("No columns contain more than 50% missing values.")
else:
    print("Columns containing more than 50% missing values:")
    print(high_missing)
```

Output:

```
No columns contain more than 50% missing values.
```

Insights:

- Each column was checked against the 50% missing-value threshold.
- Columns exceeding the threshold can be considered for removal or further investigation.
- This helps improve the quality of the dataset before analysis.

5. Insert Cleaned Data into SQLite

5.1 Create SQLite Database and Table

Description: Create an SQLite database and a table based on the cleaned dataset structure.

Code:

```
def create_sqlite_table(data):
    show_section("5. INSERT CLEANED DATA INTO SQLITE")
    show_subsection("5.1 CREATE SQLITE DATABASE AND TABLE")
    DB_DIR.mkdir(parents=True, exist_ok=True)
    connection = sqlite3.connect(SQLITE_DB)
    cursor = connection.cursor()
    # Remove old table so the program can be executed again
    cursor.execute(
        f'DROP TABLE IF EXISTS "{TABLE}"'
    )
    column_definitions = []
    for column in data.columns:
        sql_type = sqlite_type(data[column])
        safe_name = column.replace("'", '"')
        column_definitions.append(
            f'"{safe_name}" {sql_type}'
        )
    create_statement = f'''
        CREATE TABLE "{TABLE}" (
            {", ".join(column_definitions)}
        )
    '''
    cursor.execute(create_statement)
    connection.commit()
    print("SQLite database created successfully.")
    print("Database:", SQLITE_DB)
    print("Table    :", TABLE)
    return connection
```

Output:

```
SQLite database created successfully.  
Database: C:\Users\lkmuser\Project_bharath\database\chicago_crime.db  
Table    : chicago_crime
```

Insights:

- An SQLite database was created successfully.
- The chicago_crime table was created using the cleaned dataset columns.
- Appropriate SQLite data types were assigned based on the Pandas data types.

5.2 Insert Cleaned Records into SQLite

Description: Insert the cleaned crime records into the SQLite table.

Code:

```
def insert_records(data, connection):  
    show_subsection("5.2 INSERT CLEANED RECORDS INTO SQLITE")  
    cursor = connection.cursor()  
    # Make a separate copy for SQLite insertion  
    records_df = data.copy()  
    # Convert datetime columns into SQLite-compatible text  
    for column in records_df.columns:  
        if pd.api.types.is_datetime64_any_dtype(records_df[column]):  
            records_df[column] = records_df[column].dt.strftime(  
                "%Y-%m-%d %H:%M:%S")  
    # Replace missing values with None  
    records_df = records_df.astype(object)  
    records_df = records_df.where(  
        pd.notna(records_df), None)  
    # Convert NumPy values into normal Python values  
    def convert_value(value):  
        if isinstance(value, np.generic):  
            return value.item()  
        return value  
    records = []  
  
    for row in records_df.itertuples(  
        index=False,  
        name=None):  
        converted_row = tuple(convert_value(value)  
                                for value in row)  
        records.append(converted_row)  
    # Prepare column names  
    column_names = ", ".join("'" + column.replace("'", "'") + "'"  
                                for column in records_df.columns)  
    # Create placeholders  
    placeholders = ", ".join("? " for _ in records_df.columns)  
    insert_sql = f"""  
        INSERT INTO "{TABLE}"  
        ({column_names})  
        VALUES ({placeholders})  
    """  
    # Insert all rows  
    cursor.executemany(insert_sql, records)  
    connection.commit()  
    print("Cleaned records inserted successfully.")  
    print("Number of records inserted:", len(records))  
    # Verify inserted rows  
    cursor.execute(f'SELECT COUNT(*) FROM "{TABLE}"')  
    total_rows = cursor.fetchone()[0]  
    print("Total records in SQLite:", total_rows)
```

Output:

```
Cleaned records inserted successfully.  
Number of records inserted: 2000  
Total records in SQLite: 2000  
  
SQLite connection closed.
```

Insights:

- The cleaned records were successfully inserted into SQLite.
- The number of inserted records was verified after insertion.
- SQLite provides a structured way to store the processed crime data for later analysis.

6. Questions

6.1 Number of Unique Crime Types

Description: Determine the number of unique crime types present in the dataset.

Code:

```
show_subsection("6.1 UNIQUE CRIME TYPES")  
crime_column = None  
for col in data.columns:  
    name = col.strip().lower()  
    if name in [  
        "primary_type",  
        "crime_type",  
        "type"]:  
        crime_column = col  
        break  
if crime_column is not None:  
    unique_crimes = data[crime_column].nunique()  
    print(  
        "Number of unique crime types:",  
        unique_crimes)  
else:  
    print("Crime type column was not found.")
```

Output:

```
| Number of unique crime types: 16
```

Insights:

- The unique crime types were identified from the primary_type field.
- This provides an overview of the variety of crime categories present in the dataset.
- The result can be used for category-based crime analysis.

6.2 Date Anomalies and Obvious Outliers

Description: Check the crime date column for invalid or missing dates and identify the earliest and latest valid dates.

Code:

```
show_subsection("6.2 DATE ANOMALIES AND OBVIOUS OUTLIERS")
date_column = None
for col in data.columns:
    if col.strip().lower() == "date":
        date_column = col
        break
if date_column is not None:
    invalid_dates = data[date_column].isna().sum()
    print("Invalid or missing date values:", invalid_dates)
    valid_dates = data[date_column].dropna()
    if len(valid_dates) > 0:
        print("Earliest date:", valid_dates.min() )
        print("Latest date:", valid_dates.max())
        print("Date values were checked for invalid entries.")
else:
    print("Date column was not found.")
```

Output:

```
Invalid or missing date values: 0
Earliest date: 2015-01-03 23:10:00
Latest date: 2023-12-31 09:44:00
Date values were checked for invalid entries.
```

Insights:

- The dataset was checked for invalid or missing date values.
- The earliest and latest valid dates were identified.
- This confirms the time range available for further crime analysis.
- Checking date anomalies helps ensure reliable time-based analysis.

Conclusion:

The Chicago crime dataset was successfully loaded, inspected, cleaned, and processed. Missing values and categorical fields were handled appropriately, while new time-based features such as Year, Month, and DayOfWeek were generated. The processed data was also stored in SQLite, making it ready for further analysis and visualization.